
多态

复习

1. 继承的基本概念和派生类的定义
2. 继承的语法规则
3. 三种继承方式
4. 继承中的构造和析构
5. 多继承
6. 多继承构造函数
7. 多继承命名冲突
8. 菱形继承
9. 虚继承
10. 虚继承构造函数

了解

熟悉

重点

重点

重点

重点

重点

重点

重点

重点

目录

- | | |
|----------------|----|
| 1. 多态的基本概念 | 了解 |
| 2. 构成多态的条件 | 重点 |
| 3. 引用实现多态 | 熟悉 |
| 4. 虚函数 | 重点 |
| 5. 虚析构 | 重点 |
| 6. 虚函数表 | 重点 |
| 7. C++如何实现动态绑定 | 熟悉 |
| 8. 抽象基类和纯虚函数 | 重点 |

多态的基本概念

C++面向对象的三大特征为：**封装、继承和多态**，这三种机制能够有效提高程序的可读性、可扩充性和可重用性。“多态”指的是同一名字的事物可以完成不同的功能。

多态可以分为**编译时的多态和运行时的多态**。

前者主要是指函数的重载（包括运算符的重载）、对重载函数的调用，在编译时就能根据实参确定应该调用哪个函数，因此叫编译时的多态；

而后者则和继承、**虚函数**等概念有关

多态的基本概念

有虚函数，基类指针指向基类对象时就使用基类的成员（包括 成员函数 和 成员变量 ），指向派生类对象时就使用派生类的成员。换句话说，基类指针可以按照基类的方式来做事情，也可以按照派生类的方式来做事情，它有**多种形态**，或者说有**多种表现方式**，这种现象称为多态（ Polymorphism[ˌpɒliˈmɔːfɪzəm] ）。

C++ 提供多态的目的是：可以通过基类指针对所有派生类（包括直接派生和间接派生）的成员变量和成员函数**进行“全方位”的访问**，尤其是成员函数。如果没有多态，只能访问成员变量

构成多态的条件

构成多态的条件

1. 必须存在继承关系；
2. 继承关系中必须有同名的虚函数，并且它们是覆盖关系（函数原型相同）。
3. 存在基类的指针，通过该指针调用虚函数。

构成多态的条件

```
//Person类
class Person
{
public:
    Person(string name, int age):name(name),age(age)
    {
    }
    void info()
    {
        cout << "Call Person info, Name = " << this->name << " Age = " << this->age << endl;
    }
protected:
    string name;
    int age;
};
class Student:public Person
{
public:
    Student(string name, int age, float score):Person(name, age),score(score)
    {
    }
    void info()
    {
        cout << "Call Student info, Name = " << this->name << " Age = " << this->age << " Score = " << score << endl;
    }
protected:
    float score;
};
int main()
{
    Person *person = new Person("zs", 20);
    person->info();
    Student *student = new Student("ww", 21, 90);
    student->info();
    return 0;
}
```

构成多态的条件

定义了一个 Person 类和一个 Student 类，Student 类继承自 Person 类，接着，在 main 函数里面，我们分别实例化了一个 Person 对象和一个 Student 对象。

最后，分别调用了 Person 类对象的 info 方法和 Student 类对象的 info 方法，这时发现，它们各自调用了自己的 info 函数，现在，用 Student 来实例化 Person 类。

```
Call Person info, Name = zs Age = 20  
Call Student info, Name = ww Age = 21 Score = 90
```


构成多态的条件

```
int main()
{
    Person *person = new Person("zs", 20);
    person->info();
    person = new Student("ww", 21, 90);
    person->info();
    return 0;
}
```

构成多态的条件

用 Student 类实例化了 Person 类，最终调用 info 函数，此时的 **info 函数还是调用的 Person 类的**，这不是想要的效果，我们期望的是还是调用 Student 类的 info 函数。

```
Call Person info, Name = zs Age = 20  
Call Person info, Name = ww Age = 21  
-----
```

构成多态的条件

通过基类指针只能访问派生类的成员变量，但是不能访问派生类的成员函数。为了能让基类指针访问派生类的成员函数，C++ 增加了虚函数（ Virtual Function ），现在，将 info 函数声明为虚函数。

在Person类和Student类中的void info()前都加上virtual关键字.

virtual void info().....

构成多态的条件

修改代码之后，再次运行，这次使用 person 对象可以调用到 Student 实例的 info 方法了，即，通过虚函数实现了多态。

```
Call Person info, Name = zs Age = 20  
Call Student info, Name = ww Age = 21 Score = 90
```

引用实现多态

指针和引用很像，在 C++ 中，多态的实现，除了可以使用子类的指针指向父类的对象之外，还可以通过引用来实现多态，不过引用不像指针灵活，指针可以随时改变指向，而引用只能指代固定的对象，在多态性方面缺乏表现力。

引用实现多态

```
Person p1("xiaoming",100);
```

```
Student s1("xiaozhang",120,99);
```

```
Person &rPerson = p1;
```

```
Person &rStuent = s1;
```

```
rPerson.info();
```

```
rStuent.info();
```

引用类似于常量，只能在定义的同时初始化，并且以后也要从一而终，不能再引用其他数据，所以本例中必须要定义两个引用变量，一个用来引用基类对象，一个用来引用派生类对象。从运行结果可以看出，当基类的引用指代基类对象时，调用的是基类的成员，而指代派生类对象时，调用的是派生类的成员。

即，我们使用了引用实现了多态的功能。

虚函数

在 C++ 中，使用 virtual 关键字 修饰的 函数 被称为虚函数，虚函数对于 多态 具有决定性的作用，**有虚函数才能构成多态**。

什么时候需要将函数声明为虚函数，首先看 成员函数 所在的类是否会作为基类。然后看成员函数在类的继承后有无可能被更改功能，如果希望更改其功能的，一般应该将它声明为虚函数。

如果成员函数在类被继承后功能不需修改，或派生类用不到该函数，则不要把它声明为虚函数。

虚函数

上个案例中的Person类中和Student类中的的info函数，就是虚函数。必须将 info 函数声明为虚函数，不然，没办法通过子类对象指向父类成员时，访问子类对象的 info 方法。

语法

```
virtual type funcName(plist){}
```


虚函数案例

定义一个Animal类，在定义一个虚函数eat；在定义一个dog类和cat类继承Animal，通过多态调用不同的eat函数。

虚函数案例

定义一个Animal类，在定义一个虚函数eat；在定义一个dog类和cat类继承Animal，通过多态调用不同的eat函数。

```
Animal a1 = new Animal();  
a1->eat();  
a1 = new Dog();  
a1->eat();  
a1 = new Cat();  
a1->eat();
```



? 有问题吗

虚函数案例

```
class Animal{
public:
    virtual void eat(){
        cout << "Animal eat" << endl;
    }
};
class Dog:public Animal{
public:
    virtual void eat(){
        cout << "Dog eat" << endl;
    }
};
class Cat: public Animal{
    virtual void eat(){
        cout << "Cat eat" << endl;
    }
};

int main(){
    Animal *a1 = new Animal();
    a1->eat();
    a1 = new Dog();
    a1->eat();
    a1 = new Cat();
    a1->eat();
    return 0;
}
```

```
Animal a0;
Animal &a1 = a0;
a1.eat();
Dog d1;
Animal &a2 = d1;
a2.eat();
Cat c1;
Animal &a3 = c1;
a3.eat();
```

虚析构

在 C++ 中，使用 virtual 关键字 修饰的 函数 被称为 虚函数，C++ 的构造函数不可以被声明为虚函数，但析构函数可以被声明为虚函数，并且有时候必须将析构声明为虚函数。

用 C++ 开发的时候，用来做基类的类的析构函数一般都是虚函数。

虚析构函数的作用

虚析构函数是为了避免内存泄露，而且是当子类中会有指针 成员变量 时才会使用得到的。也就是说虚析构函数使得在删除指向子类对象的基类指针时可以调用子类的析构函数达到释放子类中堆内存的目的，而防止内存泄露的。

当父类的析构函数不声明成虚析构函数的时候，当子类继承父类，父类的指针指向子类时，delete掉父类的指针，只调动父类的析构函数，而不动子类的析构函数。

当父类的析构函数声明成虚析构函数的时候，当子类继承父类，父类的指针指向子类时，delete掉父类的指针，先调动子类的析构函数，再调动父类的析构函数。

虚析构函数

语法

```
virtual ~FuncName()
```

```
{}
```

虚析构函数

```
class Person
{
public:
    Person(string name, int age):name(name),age(age)
    {
    }
    ~Person(){
        cout << "call ~Person" << endl;
    }
    virtual void info()
    {
        cout << "Call Person info, Name = " << this->name << " Age = " << this->age << endl;
    }
protected:
    string name;
    int age;
};
class Student:public Person
{
public:
    Student(string name, int age, float score):Person(name, age),score(score)
    {
    }
    ~Student()
    {
        cout << "Call ~Student" << endl;
    }
    virtual void info()
    {
        cout << "Call Student info, Name = " << this->name << " Age = " << this->age << " Score = " << score << endl;
    }
protected:
    float score;
};
int main(){
    Person *p1 = new Student("zs",20,90);
    p1->info();
    delete p1;

    return 0;
}
```

虚析构函数

在 main 函数里面，使用了父类指向了子类对象，最终，释放子类对象时，调用了父类的析构函数，这样会导致，子类对象的一些数据成员没发得到释放，会造成内存泄露。

```
Call Student info, Name = zs Age = 20 Score = 90  
call ~Person  
=====
```


虚析构函数

```
class Person
{
public:
    Person(string name, int age):name(name),age(age)
    {
    }
    virtual ~Person(){
        cout << "call ~Person" << endl;
    }
    virtual void info()
    {
        cout << "Call Person info, Name = " << this->name << " Age = " << this->age << endl;
    }
protected:
    string name;
    int age;
};
class Student:public Person
{
public:
    Student(string name, int age, float score):Person(name, age),score(score)
    {
    }
    ~Student()
    {
        cout << "Call ~Student" << endl;
    }
    virtual void info()
    {
        cout << "Call Student info, Name = " << this->name << " Age = " << this->age << " Score = " << score << endl;
    }
protected:
    float score;
};
int main(){
    Person *p1 = new Student("zs",20,90);
    p1->info();
    delete p1;

    return 0;
}
```

虚析构函数

将父类的析构函数声明为了虚析构，再次运行程序。

```
Call Student info, Name = zs Age = 20 Score = 90  
Call ~Student  
call ~Person
```

虚析构函数

用 C++ 开发的时候，用来做基类的类的析构函数一般都是虚函数。

虚析构函数是为了避免内存泄露，而且是当子类中会有指针成员变量时才会使用得到的。也就是说虚析构函数使得在删除指向子类对象的基类指针时可以调用子类的析构函数达到释放子类中堆内存的目的，而防止内存泄露的。

虚函数表

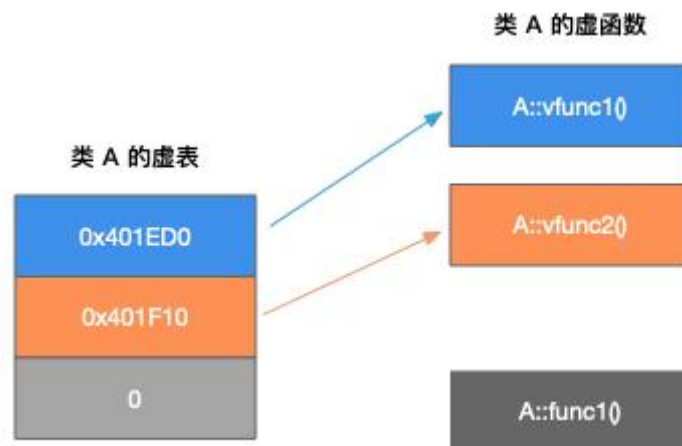
在 C++ 中，多态是由虚函数实现的，而虚函数主要是通过虚函数表（V-Table）来实现的。对象不包含虚函数表，只有虚指针，类才包含虚函数表，派生类会生成一个兼容基类的虚函数表。

如果一个类中包含虚函数（virtual 修饰的函数），那么这个类就会包含一张虚函数表，虚函数表存储的每一项是一个虚函数的地址。

虚函数表

C++ 的虚函数表如下图所示：

```
class A {  
public:  
    virtual void vfunc1();  
    virtual void vfunc2();  
    void func1();  
    void func2();  
private:  
    int m_data1, m_data2;  
};
```



虚函数表

虚表是一个指针数组，其元素是虚函数的指针，每个元素对应一个虚函数的函数指针。需要指出的是，普通的函数即非虚函数，其调用并不需要经过虚表，所以虚表的元素并不包括普通函数的函数指针。

虚表内的条目，即虚函数指针的赋值发生在编译器的编译阶段，也就是说在代码的编译阶段，虚表就可以构造出来了。

虚表是属于类的，而不是属于某个具体的对象，一个类只需要一个虚表即可。同一个类的所有对象都使用同一个虚表。

C++如何实现动态绑定

动态绑定什么时候发生？所有的工作都是由编译器在幕后完成。当我们告诉通过创建一个virtual函数来告诉编译器要进行动态绑定，那么编译器就会根据动态绑定机制来实现我们的要求，不会再执行早绑定。

编译器如何处理虚函数。当编译器发现类中有虚函数的时候，编译器会创建一张虚函数表，把虚函数的函数入口地址放到虚函数表中，并且在类中秘密增加一个指针，这个指针就是vpointer(缩写vptr)，这个指针是指向对象的虚函数表。在多态调用的时候，根据vptr指针，找到虚函数表来实现动态绑定。

C++如何实现动态绑定

动态绑定什么时候发生？所有的工作都是由编译器在幕后完成。当我们告诉通过创建一个virtual函数来告诉编译器要进行动态绑定，那么编译器就会根据动态绑定机制来实现我们的要求，不会再执行早绑定。

编译器如何处理虚函数。当编译器发现类中有虚函数的时候，编译器会创建一张虚函数表，把虚函数的函数入口地址放到虚函数表中，并且在类中秘密增加一个指针，这个指针就是vpointer(缩写vptr)，这个指针是指向对象的虚函数表。在多态调用的时候，根据vptr指针，找到虚函数表来实现动态绑定。

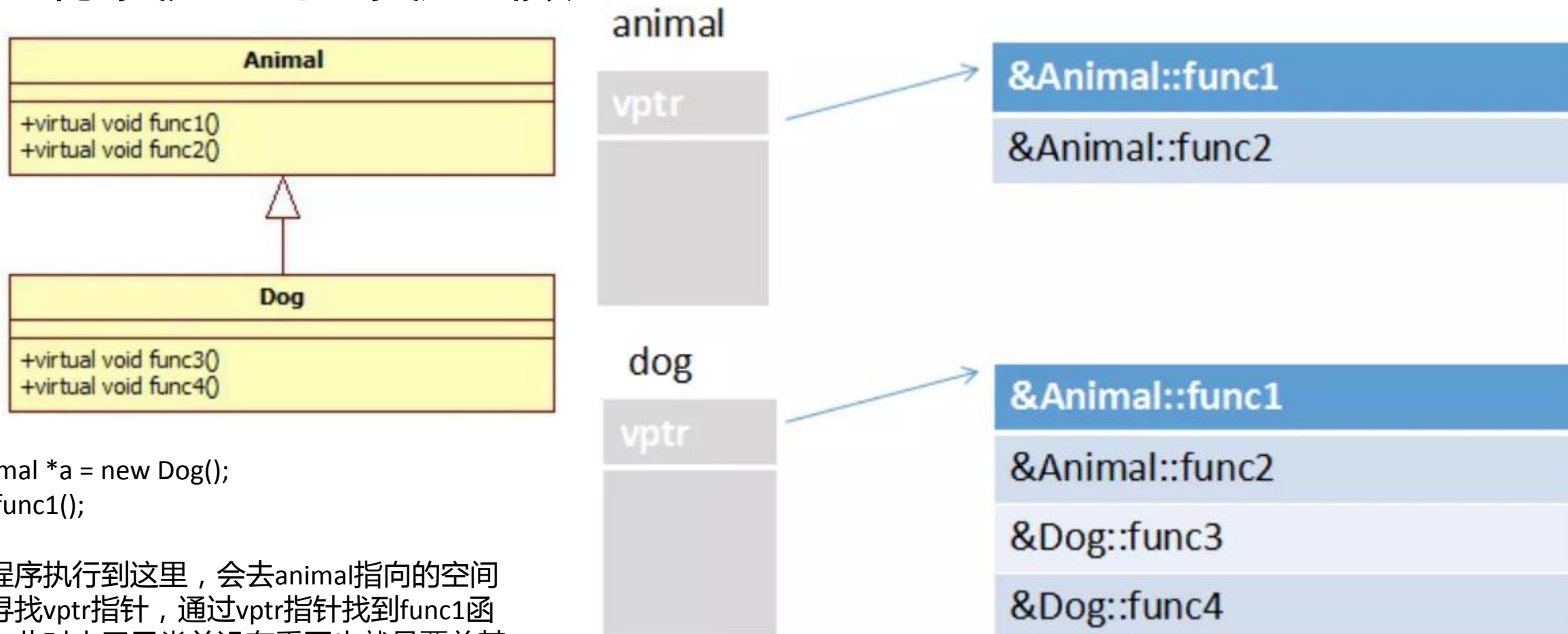
C++如何实现动态绑定

在编译阶段，编译器秘密增加了一个vptr指针，但是此时vptr指针并没有初始化指向虚函数表(vtable),什么时候vptr才会指向虚函数表？在对象构建的时候，也就是在对象初始化调用构造函数的时候。编译器首先默认会在我们所编写的每一个构造函数中，增加一些vptr指针初始化的代码。如果没有提供构造函数，编译器会提供默认的构造函数，那么就会在默认构造函数里做此项工作，初始化vptr指针，使之指向本对象的虚函数表。

起初，子类继承基类，子类继承了基类的vptr指针，这个vptr指针是指向基类虚函数表，当子类调用构造函数，使得子类的vptr指针指向了子类的虚函数表。

C++如何实现动态绑定

当子类无重写基类虚函数时



Animal *a = new Dog();
a->func1();

当程序执行到这里，会去animal指向的空间中寻找vptr指针，通过vptr指针找到func1函数，此时由于子类并没有重写也就是覆盖基类的func1函数，所以调用func1时，仍然调用的是基类的func1.

当子类无重写基类虚函数时代码验证

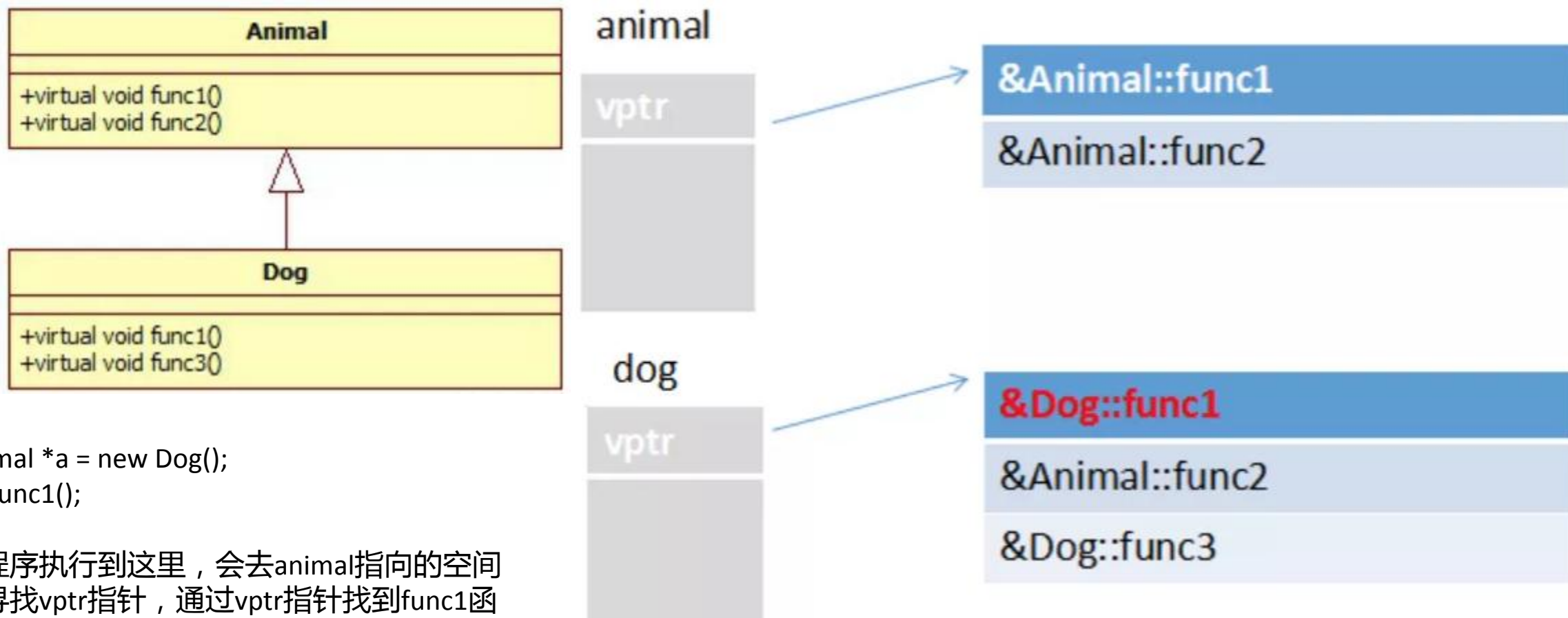
```
class Animal{
public:
    virtual void func1(){
        cout << "Animal func1" << endl;
    }
    virtual void func2(){
        cout << "Animal func1" << endl;
    }
};
class Dog:public Animal{
public:
    virtual void func3(){
        cout << "Dog func3" << endl;
    }
    virtual void func4(){
        cout << "Dog func4" << endl;
    }
};

int main(){
    Animal *a=new Dog;
    a->func1();

    return 0;
}
```

C++如何实现动态绑定

当子类重写基类虚函数时



```
Animal *a = new Dog();  
a->func1();
```

当程序执行到这里，会去animal指向的空间中寻找vptr指针，通过vptr指针找到func1函数，由于子类重写基类的func1函数，所以调用func1时，调用的是子类的func1.

当子类重写基类虚函数时时代码验证

```
class Animal{
public:
    virtual void func1(){
        cout << "Animal func1" << endl;
    }
    virtual void func2(){
        cout << "Animal func1" << endl;
    }
};
class Dog:public Animal{
public:
    virtual void func1(){
        cout << "Dog func1" << endl;
    }
    virtual void func4(){
        cout << "Dog func4" << endl;
    }
};

int main(){
    Animal *a=new Dog;
    a->func1();

    return 0;
}
```

抽象基类和纯虚函数

在设计时，常常希望基类仅仅作为其派生类的一个接口。这就是说，仅想对基类进行向上类型转换，使用它的接口，而不希望用户实际地创建一个基类的对象。要做到这点，可以在基类中加入至少一个纯虚函数，来使基类成为抽象类。纯虚函数使用关键字virtual，并且在其后面加上=0。如果某人试着生成一个抽象类的对象，编译器会制止他，这个工具允许生成特定的设计。

当继承一个抽象类时，必须实现所有的纯虚函数，否则继承出的类也将是一个抽象类。创建一个纯虚函数允许在接口中放置成员函数，而不一定要提供一段可能对这个函数毫无意义的代码。同时，纯虚函数要求出的类对它提供一个定义。纯虚函数总是变成“哑”函数。

建立公共接口，也就是纯虚函数抽象类。它能对于每个不同的子类有不同的表示，它建立一个基本的格式。

抽象基类和纯虚函数

纯虚函数

虚函数： `virtual type funcName(plist) {}`

纯虚函数： `virtual type funcName(plist) = 0;`

抽象基类

基类中添加了至少一个纯虚函数的基类称为抽象基类。

抽象基类和纯虚函数 案例

抽象基类是泡饮品的抽象操作

子类 泡咖啡是 泡饮品的具体操作

子类 泡茶是 泡饮品的具体操作

泡茶和泡咖啡通过继承泡饮品的共用操作，完成实现。

抽象基类和纯虚函数 案例代码

//抽象基类，制作饮品

class AbstractDrinking

{

public:

 //烧水

 virtual void Boil() = 0;

 //泡

 virtual void Brow() = 0;

 //倒入杯子

 virtual void PourInCup() = 0;

 //辅料

 virtual void PutSomething() = 0;

 //规定流程

 void MakeDrink()

 {

 Boil();

 Brow();

 PourInCup();

 PutSomething();

 }

};

抽象基类和纯虚函数 案例代码

//子类制作咖啡

class Coffee : public AbstractDrinking

{

public:

 //烧水

 virtual void Boil()

 {

 cout << "煮山泉" << endl;

 }

 //泡

 virtual void Brow()

 {

 cout << "泡咖啡" << endl;

 }

 //倒入杯子

 virtual void PourInCup()

 {

 cout << "咖啡倒入杯子" << endl;

 }

 //辅料

 virtual void PutSomething()

 {

 cout << "加牛奶" << endl;

 }

};

抽象基类和纯虚函数 案例代码

//子类 泡茶

class Tea : public AbstractDrinking

{

public:

 //烧水

 virtual void Boil()

 {

 cout << "煮白开水" << endl;

 }

 //泡

 virtual void Brow()

 {

 cout << "泡茶" << endl;

 }

 //倒入杯子

 virtual void PourInCup()

 {

 cout << "茶倒入杯子" << endl;

 }

 //辅料

 virtual void PutSomething()

 {

 cout << "加盐" << endl;

 }

};

抽象基类和纯虚函数 案例代码

//业务函数

```
void DoBussiness(AbstractDrinking* drink)
{
    drink->MakeDrink();
    delete drink;
}
```

```
int main()
{
    DoBussiness(new Coffee);
    cout << endl;
    DoBussiness(new Tea);

    return 0;
}
```

知识点总结

- | | |
|----------------|----|
| 1. 多态的基本概念 | 了解 |
| 2. 构成多态的条件 | 重点 |
| 3. 引用实现多态 | 熟悉 |
| 4. 虚函数 | 重点 |
| 5. 虚析构 | 重点 |
| 6. 虚函数表 | 重点 |
| 7. C++如何实现动态绑定 | 熟悉 |
| 8. 抽象基类和纯虚函数 | 重点 |

作业

1. 什么是虚函数？虚函数有哪些特点？

下节课预告

综合案例5